Indexing and Query Optimization

How a DBMS finds data quickly

1. What is an Index?

An index is an auxiliary data structure that lets the DBMS locate rows without scanning the whole table. It works like the index at the back of a book: instead of reading every page, you look up a key and jump straight to the right location. An index stores search-key values together with pointers to the matching records on disk.

2. Why Indexes Matter

- Disk I/O is the main cost of a query, and a full table scan reads every block.
- A good index turns a search over millions of rows into a handful of block reads.
- Indexes speed up WHERE filters, joins, ORDER BY and GROUP BY.
- The price is extra storage, and slower INSERT, UPDATE and DELETE because the index must be kept in sync.

3. Kinds of Indexes

- **Primary index:** built on the ordered primary key of a sorted file.
- **Clustering index:** built on a non-key field; rows with the same value are stored together.
- **Secondary index:** built on any other field; the file itself is not sorted by it, so the index must be dense.
- **Dense index:** one entry for every search-key value in the file.
- **Sparse index:** one entry per block; smaller, but needs the file to be sorted on the key.

4. B-Tree and B+ Tree

Most relational systems use the B+ tree as the default index. It is a balanced multi-way tree, so every leaf is at the same depth and lookups take O(log n) block reads. In a B+ tree, internal nodes hold only keys for routing, all the record pointers sit in the leaf level, and the leaves are linked in a list. That linked list makes range queries such as *salary BETWEEN 30000 AND 50000* very cheap.

5. Hash Index

A hash index applies a hash function to the key and goes straight to a bucket. It is excellent for equality lookups (*id = 42*) but cannot help with range queries or sorting, because hashing destroys the order of the keys.

6. Choosing the Right Index

Index type	Best for	Weak at
B+ tree	Equality, range, ORDER BY	Slightly more costly to maintain
Hash	Exact-match lookups	Range queries, sorting
Bitmap	Low-cardinality columns (gender, status)	Frequently updated columns
Composite	Queries filtering on several columns	Queries that skip the leading column

7. Stages of Query Processing

- **Parsing:** check the SQL syntax, resolve table and column names, and build an internal tree.
- **Optimization:** generate several equivalent execution plans and pick the cheapest.
- **Execution:** the chosen plan is run by the execution engine, and results are returned to the user.

8. Cost-Based Optimization

A cost-based optimizer estimates the cost of each candidate plan using statistics held in the catalog: number of rows, number of blocks, distinct values per column and the distribution of values. The estimated fraction of rows a condition keeps is called its **selectivity**. A highly selective condition (few matching rows) is a good reason to use an index; a condition that matches most of the table is usually faster with a plain scan.

9. Heuristic Rules of Optimization

- Perform selections as early as possible so fewer rows flow upward.
- Perform projections early to drop unneeded columns.
- Replace a Cartesian product followed by a selection with a join.
- Join the smallest intermediate results first, and choose the join order carefully.
- Prefer index scans for highly selective conditions.

10. Join Algorithms

Algorithm	Idea	Good when
Nested loop	For each outer row, scan the inner table	One table is tiny or indexed
Sort-merge	Sort both inputs, then merge	Inputs already sorted
Hash join	Build a hash table on one input, probe with the other	Large unsorted equality joins

11. Practical Tips

- Use the EXPLAIN command to see the plan the optimizer chose.
- Do not index tiny tables or columns that are updated constantly.
- Put the most selective column first in a composite index.